Como o Claude Code funciona

Autenticação, crédito de API, arquitetura, custo e segurança

Material técnico para o Lucas

Conferido na documentação oficial (code.claude.com/docs) em 01/11/2026

COBRANÇA

Assinatura e crédito de API são dois sistemas separados

	Assinatura (Pro / Max / Ultra)	Crédito de API (Console)
Onde fica	claude.ai	platform.claude.com, aba Billing
Como cobra	Mensalidade fixa, uso dentro de limites do plano	Por token consumido, abatido do saldo comprado
Como o Code entra	/login com a conta claude.ai	/login pela conta do Console, ou ANTHROPIC_API_KEY
Quando acaba	Estoura o limite (janela de 5 horas e semanal)	Saldo chega a zero ou bate o limite de gasto do workspace
Um paga o outro?	Não	Não

Resumo: o crédito de US$ 200 só é gasto quando o Claude Code está autenticado pela conta do Console ou por uma chave de API dessa organização.

Fonte: code.claude.com/docs (authentication, costs)

AUTENTICAÇÃO

Qual credencial o Code usa quando há mais de uma

Ordem	Credencial
1	Provedor de nuvem (Bedrock, Vertex, Foundry)
2	ANTHROPIC_AUTH_TOKEN (token Bearer)
3	ANTHROPIC_API_KEY (no modo interativo pede aprovação uma vez)
4	apiKeyHelper (script que devolve a chave)
5	CLAUDE_CODE_OAUTH_TOKEN (gerado por claude setup-token, vale 1 ano)
6	Perfis e federação (ANTHROPIC_PROFILE)
7	Login da assinatura (/login)

Pegadinha

Com assinatura ativa e ANTHROPIC_API_KEY definida, a chave passa a valer após a aprovação. Se a organização da chave estiver desativada, a sessão falha. Solução oficial: unset ANTHROPIC_API_KEY.

Conferir o que está ativo

Digite /status. Mostra o método de login e, se houver login e chave juntos, marca qual não está em uso.

Fonte: code.claude.com/docs/en/authentication, seção Authentication precedence

ARQUITETURA

O núcleo: um loop de três fases, repetido até terminar

1. Reunir contexto

Lê arquivos, busca no código, roda comandos de leitura

2. Agir

Edita arquivos, executa comandos, chama ferramentas

3. Verificar

Roda testes e confere o resultado; se falhou, volta à fase 1

Modelo e harness são partes diferentes. O modelo raciocina; a camada em volta fornece as ferramentas e gerencia o contexto.

Cinco categorias de ferramentas: arquivos, busca, execução, web e inteligência de código.

Cada chamada de ferramenta gera uma nova requisição ao modelo. Isso pesa no custo (slide 12).

Fonte: code.claude.com/docs/en/how-claude-code-works

ARQUITETURA

Contexto: a memória de trabalho do Code

O que ocupa a janela: histórico da conversa, conteúdo de arquivos lidos, saída de comandos, CLAUDE.md, memória automática, skills carregadas e instruções do sistema.

Compactação automática: quando enche, o Code resume a conversa. Instruções antigas podem se perder no resumo.

/compact [foco]: você controla o que o resumo preserva. Use em pausas naturais.

/clear: começa conversa nova com contexto vazio. Use ao trocar de assunto.

Memória automática: as primeiras 200 linhas ou 25 KB do arquivo MEMORY.md carregam em toda sessão.

Editar CLAUDE.md no meio da sessão não tem efeito até /clear, /compact ou reiniciar.

Fonte: code.claude.com/docs/en/how-claude-code-works, commands

ARQUITETURA

CLAUDE.md: instruções que carregam em toda sessão

Escopo	Onde fica	Para quê
Organização	Definido pelo administrador	Regras que valem para todos
Usuário	~/.claude/CLAUDE.md	Suas preferências em todos os projetos
Projeto	CLAUDE.md na raiz do repositório	Regras do projeto, vai no Git

Tamanho: a doc recomenda até 200 linhas. Texto longo gasta contexto em toda sessão.

/init gera um CLAUDE.md inicial analisando o projeto.

CLAUDE.md é pedido, não trava. Para obrigar algo de forma determinística, use hook (slide 8) ou regra de permissão.

Fonte: code.claude.com/docs/en/memory, features-overview

SEGURANÇA

Modos de permissão: quanto o Code faz sem perguntar

Modo	Comportamento
default (manual)	Pede aprovação para editar arquivos e rodar comandos
acceptEdits	Aceita edições de arquivo sozinho; comandos ainda pedem
plan	Só planeja e lê; não altera nada até você aprovar o plano
auto	Um classificador de modelo bloqueia o que parece arriscado, em vez de perguntar
dontAsk	Não pergunta; só executa o que as regras permitem
bypassPermissions	Sem prompts. Só em ambiente isolado (VM ou contêiner)

Alterna com Shift+Tab ou --permission-mode. Regra deny vale em todos os modos, inclusive bypass. Escrita em .git, .claude e .bashrc nunca é aprovada automaticamente (exceto em bypass).

Fonte: code.claude.com/docs/en/permission-modes

ARQUITETURA

Quatro formas de estender o Code

Recurso	O que é	Quando usar
Subagente	Arquivo Markdown em .claude/agents/ com prompt e ferramentas próprios; janela de contexto separada	Tarefa grande de leitura/pesquisa que não deve poluir a conversa principal
Skill	Fluxo ou conhecimento em Markdown; só a descrição fica no contexto, o resto carrega sob demanda	Procedimento que você repete (ex.: fechar sessão)
Hook	Comando disparado em evento (antes de usar ferramenta, ao iniciar sessão)	Regra que precisa ser cumprida sempre, sem depender do modelo
Plugin	Pacote com skills, hooks, subagentes e MCP, distribuído por marketplace	Compartilhar um conjunto pronto

Fonte: code.claude.com/docs/en/features-overview, sub-agents, hooks-guide

ARQUITETURA

MCP: conectar o Code a serviços externos

Escopo	Onde fica	Observação
Local (padrão)	~/.claude.json	Só você, só neste projeto
Project	Arquivo .mcp.json na raiz	Vai no Git; cada pessoa precisa aprovar
User	~/.claude.json	Vale em todos os seus projetos

Precedência: Local vence Project, que vence User.

Adicionar: claude mcp add --transport http <nome> <url>

Custo: cada servidor MCP conectado ocupa contexto e conectar ou remover no meio da sessão invalida o cache. Desative os que não usa.

Fonte: code.claude.com/docs/en/mcp

CONFIGURAÇÃO

settings.json: quatro escopos, com ordem de prioridade

Prioridade	Escopo	Arquivo
1 (maior)	Gerenciado (organização)	Definido pelo administrador
2	Flag --settings	Na linha de comando
3	Projeto local	Pasta .claude, arquivo settings.local.json (fora do Git)
4	Projeto compartilhado	Pasta .claude, arquivo settings.json (vai no Git)
5 (menor)	Usuário	~/.claude/settings.json

Listas como permissions.allow se somam entre escopos, não se substituem.

Sessões na nuvem não leem o seu ~/.claude/settings.json nem o settings.local.json.

Fonte: code.claude.com/docs/en/settings

EXECUÇÃO

Onde o Code roda: terminal local, nuvem ou worktree

Local (CLI / desktop)

Roda na sua máquina, com seus arquivos e suas configurações. Usa o login da assinatura ou da API conforme a precedência.

Nuvem (web, app)

VM gerenciada pela Anthropic (ou ambiente próprio). Continua com o laptop fechado. Consome o limite do plano, sem cobrança extra por VM. Iniciar: claude --cloud "tarefa". Trazer para o terminal: --teleport.

Worktree

claude --worktree <nome> cria uma cópia isolada do repositório em .claude/worktrees/<nome>, com branch própria. Permite trabalhos em paralelo sem conflito. O arquivo .worktreeinclude copia itens ignorados, como .env.

Fonte: code.claude.com/docs/en/claude-code-on-the-web, worktrees

CUSTO

Como o custo se forma

Por token. A conversa inteira é reenviada a cada requisição. Quanto maior o contexto, mais cada passo custa.

Referência da própria doc: cerca de US$ 13 por desenvolvedor por dia ativo, e 90% dos usuários abaixo de US$ 30 por dia. É média, não garantia.

/usage mostra uso da sessão e do plano. O valor é estimado a preço de lista. A fatura real está no Usage do Console. /cost é apelido de /usage.

Prompt caching automático: reler o que já foi enviado custa menos. Validade de 5 minutos, ou 1 hora na assinatura dentro do plano.

O cache é perdido ao: trocar de modelo, mudar o effort, conectar ou remover MCP, usar /compact, atualizar o Code.

Raciocínio (thinking) conta como token de saída e não pode ser desligado nos modelos 5.5.

Fonte: code.claude.com/docs/en/costs, prompt-caching, model-config

CUSTO

Como gastar menos sem perder qualidade

Prática	Por que ajuda
/clear ao trocar de tarefa	Contexto vazio, requisições mais baratas
/compact em pausas naturais	Resume em vez de arrastar tudo
CLAUDE.md curto; procedimentos em skills	Skill só carrega quando usada
Sonnet no dia a dia, Opus para decisões de arquitetura	Recomendação da própria doc; /model troca
/effort baixo em tarefa simples	Menos raciocínio, menos tokens
Subagente para pesquisa pesada	Leitura grande fica fora da conversa principal. Mas gasta tokens próprios
Desativar MCP que não usa	Menos contexto fixo por requisição

Fonte: code.claude.com/docs/en/costs, seção Reduce token usage

SEGURANÇA

Segurança: o que protege e o que não protege

Sandbox (/sandbox)

Isola apenas comandos Bash. Escrita só no diretório de trabalho. Rede passa por proxy com lista de domínios (vazia por padrão). Funciona em macOS, Linux e WSL2. Windows nativo roda sem sandbox.

Fica fora do sandbox: Read, Edit, Write, WebFetch, WebSearch, hooks e MCP locais.

Leitura é ampla por padrão, inclusive ~/.ssh e ~/.aws.

Chaves e credenciais

Chave nunca no repositório. Use variável de ambiente, apiKeyHelper ou o chaveiro do sistema.

Exemplo oficial de bloqueio: deny em Read(./.env) e Read(./.env.*).

Preferências pessoais vão em settings.local.json.

Checkpoints desfazem arquivos, mas não ações remotas (banco, deploy).

Na nuvem, o envio de repositório local exclui .env, .pem, id_rsa e .tfvars.

Fonte: code.claude.com/docs/en/sandboxing, security, settings

APLICAÇÃO

Na prática, para o seu caso

1. Para gastar o crédito de US$ 200: faça /login pela conta do Console no Claude Code local, ou crie uma chave na mesma organização do crédito e use ANTHROPIC_API_KEY. Confira com /status.

2. Defina um limite de gasto no workspace do Console antes de usar. Evita que um script em loop esvazie o saldo.

3. Copiloto Clínico: se o app chamar o Claude por dentro, a chave fica no servidor, nunca no GitHub, e nenhum dado de paciente vai em prompt, arquivo ou exemplo.

4. Este tipo de sessão (web/nuvem) usa a assinatura, não o crédito de API.

5. Dia a dia: Sonnet com /clear entre tarefas costuma bastar; reserve Opus para decisão de arquitetura.

Fonte: Síntese; itens de custo conforme os slides anteriores

LIMITES DESTA APRESENTAÇÃO

O que a documentação não permitiu confirmar

Saldo pré-pago do Console: recarga automática e o que acontece com saldo zero.

Valores exatos dos limites de gasto por workspace.

Preços e cotas de Pro, Max e Ultra: ver claude.com/pricing.

Ordem exata entre regras allow, ask e deny na página de permissões.

Lista completa de eventos de hook e códigos de saída.

Lista completa de ferramentas (Grep e Glob por nome).

Seu saldo, validade e organização do crédito: só aparecem no seu Console.

Fonte: Consulta de 01/11/2026